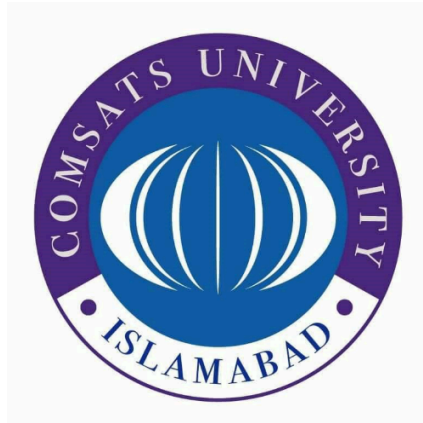




PARALLEL & DISTRIBUTED COMPUTING

LAB ASSIGNMENT #5

PREPARED BY

M Mukaram Awan
SP22-BCS-141

SUBMITTED TO

Sir Akhzar Nazeer

```

// File: parallel_sum.c
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char* argv[]) {
    // --- Initialization ---
    int rank, size;
    const long N = 10000000; // Vector size as per assignment [cite: 29]
    double *A = NULL;
    double local_sum = 0.0, global_sum = 0.0;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    // Calculate how many elements each process will handle
    long local_n = N / size;
    // Allocate memory for the local sub-vector on each process
    double *local_A = (double*)malloc(local_n * sizeof(double));

    // --- Data Preparation (Root Process Only) ---
    if (rank == 0) {
        printf("Running with %d processes.\n", size);
        A = (double*)malloc(N * sizeof(double));
        // Task 1: Initialize the large vector A [cite: 29]
        for (long i = 0; i < N; i++) {
            A[i] = i + 1;
        }
    }

    // --- Data Distribution ---
    // Task 2: Scatter the vector A from the root process to all processes [cite: 32]
    // Each process receives a chunk of size 'local_n' into its 'local_A' buffer.
    MPI_Scatter(A, local_n, MPI_DOUBLE, local_A, local_n, MPI_DOUBLE, 0,
MPI_COMM_WORLD);

    // --- Local Computation ---

```

```

// Task 3: Each process computes the sum of its local sub-vector [cite: 35]
for (long i = 0; i < local_n; i++) {
    local_sum += local_A[i];
}
// printf("Process %d calculated local_sum = %f\n", rank, local_sum); // Uncomment for
debugging

// --- Reduction (Aggregation) ---
// Task 4: Combine (reduce) all 'local_sum' values into 'global_sum' on the root process
[cite: 40]
// The operation used to combine them is MPI_SUM.
MPI_Reduce(&local_sum, &global_sum, 1, MPI_DOUBLE, MPI_SUM, 0,
MPI_COMM_WORLD);

// --- Display Results (Root Process Only) ---
if (rank == 0) {
    // Task 5: The root process prints the final result and verifies it [cite: 42]
    // Use (double)N to prevent potential integer overflow for very large N
    double expected_sum = (double)N * (N + 1) / 2.0;
    printf("\n--- Results ---\n");
    printf("Total Sum    = %.0f\n", global_sum);
    printf("Expected Sum  = %.0f\n", expected_sum);
    printf("Difference   = %.5f\n", expected_sum - global_sum);
}

// --- Cleanup ---
if (rank == 0) {
    free(A);
}
free(local_A);

MPI_Finalize();
return 0;
}

```

1. What happens if the vector size N is not divisible by the number of processes?

If N is not evenly divisible by the number of processes (size), the integer division N / size will truncate the result. This means the chunks distributed by `MPI_Scatter` will not cover the entire vector, and the elements at the end of the original vector A will be ignored, leading to an incorrect final sum.

2. How can you modify the program to handle uneven partitions?

You should use the function `MPI_Scatterv`. Unlike `MPI_Scatter` which sends equal-sized chunks, `MPI_Scatterv` allows you to define an array of chunk sizes and an array of starting positions (displacements). This gives you full control to distribute the vector even when the partitions are uneven.

3. How would performance differ between using MPI_Reduce vs. MPI_Gather + local summation?

`MPI_Reduce` is significantly more efficient and scalable.

- `MPI_Reduce`: Performs the summation in a highly optimized way (often using a tree-based algorithm) as the data is being collected across the network. It only needs to send one final number to the root.
- `MPI_Gather`: All partial sums are sent individually to the root process. The root then has to perform a final loop to sum them up. This causes a bottleneck at the root process and involves more data transfer.

4. How could this same approach be extended to matrix summation or averaging?

The approach is very similar. For a matrix, you could treat it as a single large 1D array and use the exact same logic. Alternatively, you could scatter entire rows or blocks of rows to different processes. Each process would calculate its local sum (sum of its assigned rows), and then you would use `MPI_Reduce` to get the final global sum. For the average, the root process would simply divide this global sum by the total number of elements in the matrix.